

Inheritance in Python

Track: Data Science | Level 1 - Day 14 | Tools: Python 3, Jupyter Notebook

Level 1 - Day 14: Inheritance in Python

Track: Data Science

Task: Task 14 - Inheritance and Code Reusability

Objective: Master single, multiple, and hierarchical inheritance, calling `super()` constructors, method overriding, and organizing reusable data modeling architectures.

1. Quick Reference & Core Concepts

Inheritance Type	Syntax Pattern	Data Engineering Benefit
Single Inheritance	<code>class Child(Parent):</code>	Specialization of a generic data entity
<code>super()</code> Constructor	<code>super().__init__(args)</code>	Preserves parent initialization while extending fields
Method Overriding	<code>def evaluate(self): ...</code>	Customizing behaviors for specific subclasses
Hierarchical	<code>class A; class B(A); class C(A)</code>	Common base features across distinct dataset entities
Multiple Inheritance	<code>class Child(ParentA, ParentB):</code>	Mixins for logging, serialization, and metrics

2. Practical Code Examples

In [code]

```
# Example 1: Single Inheritance with super()
class Employee:
    def __init__(self, emp_id, name, base_salary):
        self.emp_id = emp_id
        self.name = name
        self.base_salary = base_salary

    def get_details(self):
        return f"ID: {self.emp_id} | Name: {self.name} | Salary: ${self.base_salary:,"

class DataScientist(Employee):
    def __init__(self, emp_id, name, base_salary, programming_languages):
        super().__init__(emp_id, name, base_salary)
        self.languages = programming_languages

ds = DataScientist(201, "Neha", 110000, ["Python", "SQL", "R"])
print(f"Ex 1 - {ds.get_details()} | Skills: {' | '.join(ds.languages)}")
```

In [code]

```
# Example 2: Method Overriding for Dynamic Compensation
class RegularEmployee(Employee):
    def calculate_bonus(self):
        return self.base_salary * 0.05

class SeniorDataScientist(Employee):
    def calculate_bonus(self):
        return self.base_salary * 0.18 # Overridden bonus formula

reg = RegularEmployee(101, "Karan", 50000)
sen = SeniorDataScientist(102, "Aditi", 120000)
print(f"Ex 2 - Regular Bonus: ${reg.calculate_bonus():.2f}")
print(f"Ex 2 - Senior DS Bonus: ${sen.calculate_bonus():.2f}")
```

In [code]

```
# Example 3: Checking Inheritance with isinstance() and issubclass()
print(f"Ex 3 - Is sen an instance of Employee? {isinstance(sen, Employee)}")
print(f"Ex 3 - Is SeniorDataScientist a subclass of Employee? {issubclass(SeniorDataScientist, Employee)}")
```

3. Applied Data Science Pipeline

Applied Data Pipeline: Hierarchical Dataset Loader Architecture

In [code]

```
class BaseDatasetLoader:
    def __init__(self, source_name):
        self.source_name = source_name

    def load(self):
        raise NotImplementedError

class CSVDatasetLoader(BaseDatasetLoader):
    def load(self):
        return f"[CSVLoader] Parsing comma-separated tabular records from {self.source_name}"

class JSONDatasetLoader(BaseDatasetLoader):
    def load(self):
        return f"[JSONLoader] Ingesting key-value nested objects from {self.source_name}"

loaders = [CSVDatasetLoader("students.csv"), JSONDatasetLoader("telemetry.json")]
for l in loaders:
    print(l.load())
```